UNIVERSIDAD NACIONAL DE CÓRDOBA
FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES



Programación Concurrente y Paralela
Trabajo Práctico N°1

***Docentes*:** Ing. Luis Orlando Ventre, Ing. Mauricio Ludemann y Aspirante Docente Ing. Agustin Carranza
Año: 2026
Semestre: Segundo Semestre

Apellido y Nombre: *Bastía, Álvaro*
DNI: *44.783.012*
Apellido y Nombre: *Britez, Carlos*
DNI: *44.221.771*
Apellido y Nombre: *Género, Sol*
DNI: *46.541.859*
Apellido y Nombre: *Tarrago, Emilio*
DNI: *44.198.050*
Apellido y Nombre: *Usandivaras, Hebe Azul*
DNI: *44.768.075*

Índice:

Introducción: 3
Consigna: 3
Procedimiento: 3
Diagrama de clases: 3
Diagrama de secuencia: 5
Código: 7
PrinterFarm: 7
OrderSource: 8
ValidationWorker: 9
Resultados: 13
elementos.csv: 13
eventos.csv: 14
resumen.properties: 14
Conclusión: 15

Introducción:

En el presente documento se encuentra registrada la información referente al Trabajo Práctico N° 1 de Programación Concurrente y Paralela. El mismo engloba los temas vistos en la primera mitad del cuatrimestre.

Consigna:

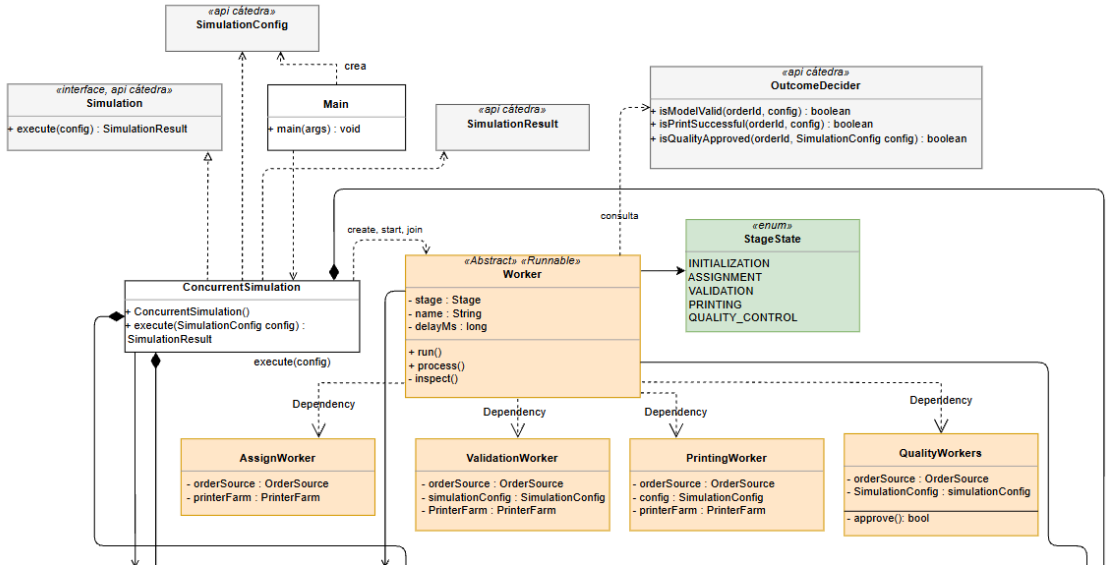
Desarrollar un sistema concurrente que simule una granja industrial de impresión 3D. El mismo recibe órdenes de fabricación y, luego de validar sus modelos, les asigna una impresora. Luego de ejecutadas las impresiones, se controla la calidad de las piezas obtenidas.

Todas las etapas deben funcionar de manera concurrente y compartir los recursos de manera segura. El sistema debe procesar todas las órdenes, conservar la consistencia de sus estados y finalizar sin dejar hilos activos.

Procedimiento:

En primer lugar, se busca plantear la solución de la consigna diseñando un diagrama de clases, el cual poseerá las entidades y métodos a implementar, y un diagrama de secuencia, que plasmará la línea de vida de los objetos y el flujo de control del sistema.

Diagrama de clases:



En esta primera parte del diagrama se ven representadas las clases dadas por la cátedra y cómo se relacionan por la clase Main, donde se inicializa la configuración y se arranca el proceso, y ConcurrentSimulation, la cual implementa la interfaz Simulation y es la clase coordinadora que se encarga de instanciar, inicializar, arrancar y esperar a que terminen (join) los diferentes hilos trabajadores (workers).

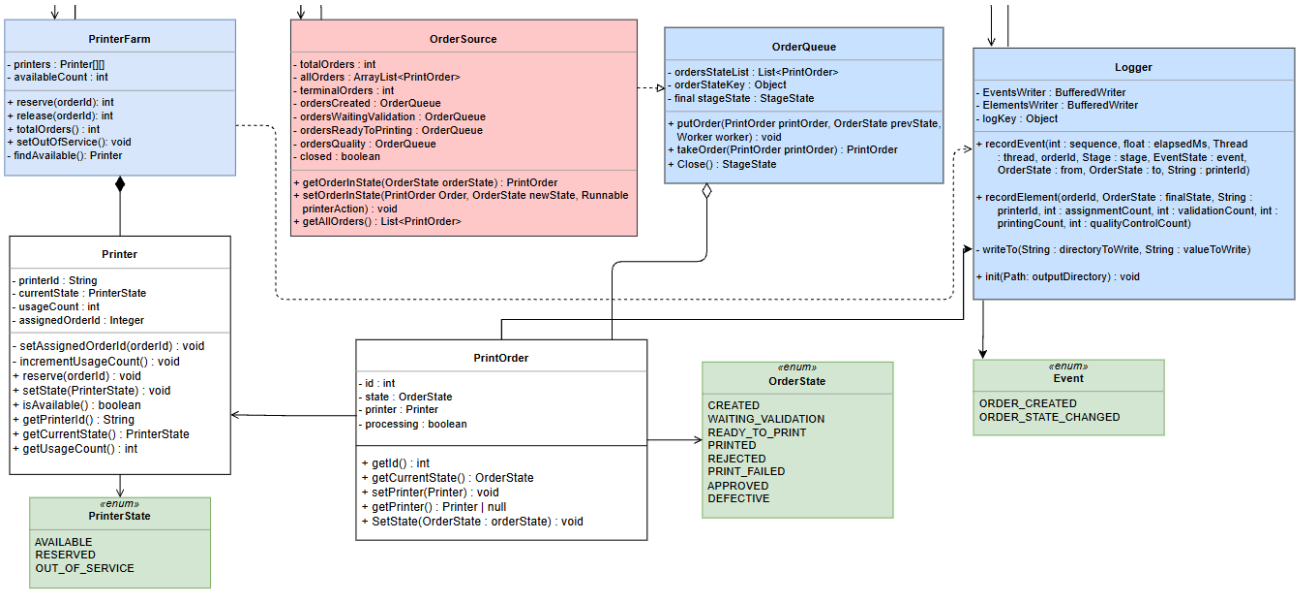
El diagrama utiliza una clase abstracta llamada Worker que implementa Runnable (cada worker se ejecutará en su propio hilo de ejecución), y de ella heredan 4 tipos de hilos que representan las etapas del proceso de fabricación:

- AssignWorker: asigna una impresora a las órdenes.

- ValidationWorker: valida el modelo 3D antes de imprimirlo.
- PrintingWorker: simula la impresión.
- QualityWorker: representa el control de calidad de las piezas.

Todos estos hilos dependen de los recursos compartidos:

- Las impresoras de PrinterFarm.
- Las listas de órdenes de OrderQueue.
- Los registros del Logger.

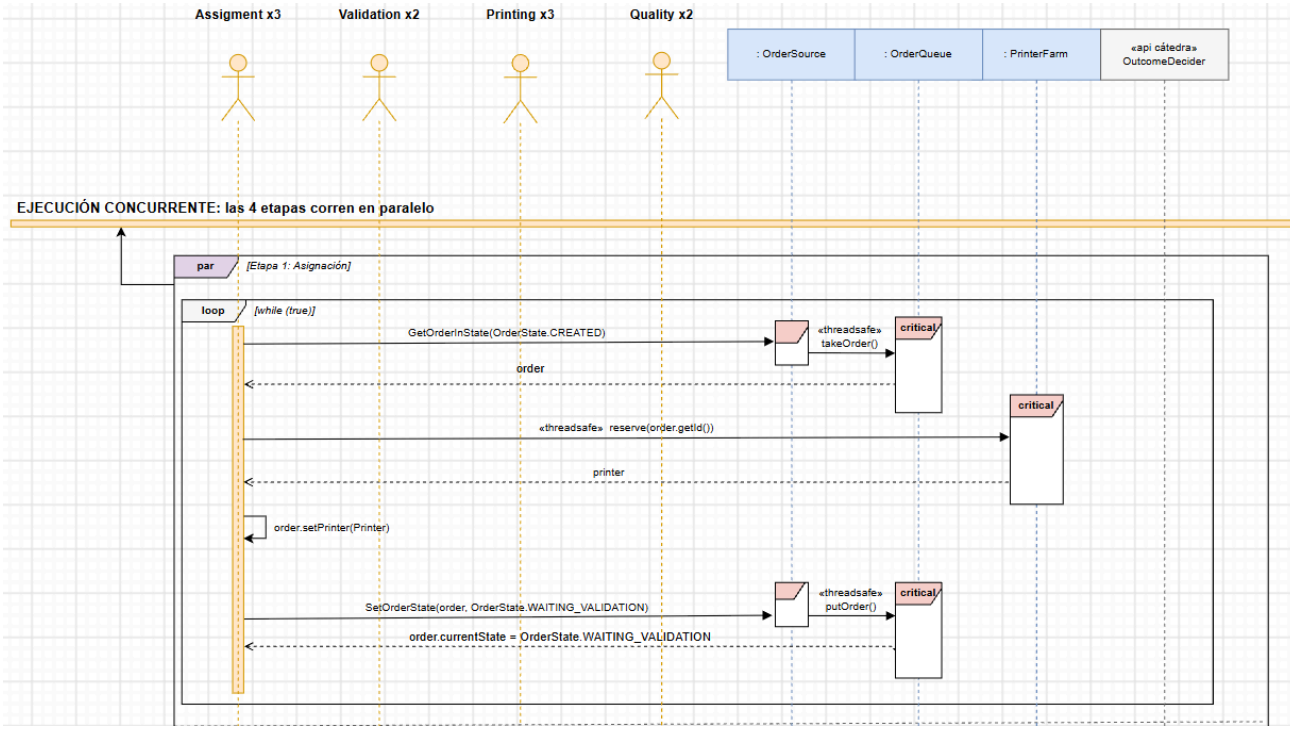


Por otro lado, en la segunda parte del diagrama, se logran apreciar en color azul las clases que manejan la concurrencia y secciones críticas, las cuales plantean una estructura de control de estado y colas para evitar las condiciones de carrera. OrderSource (en rojo) es un clasificador de órdenes (PrintOrder) y los contenedores son instancias de OrderQueue; PrinterFarm es la granja que actúa como administrador de recursos y controla el arreglo de objetos Printer; y Logger es quien registra los eventos por cambio de estado y una fila por orden de fabricación.

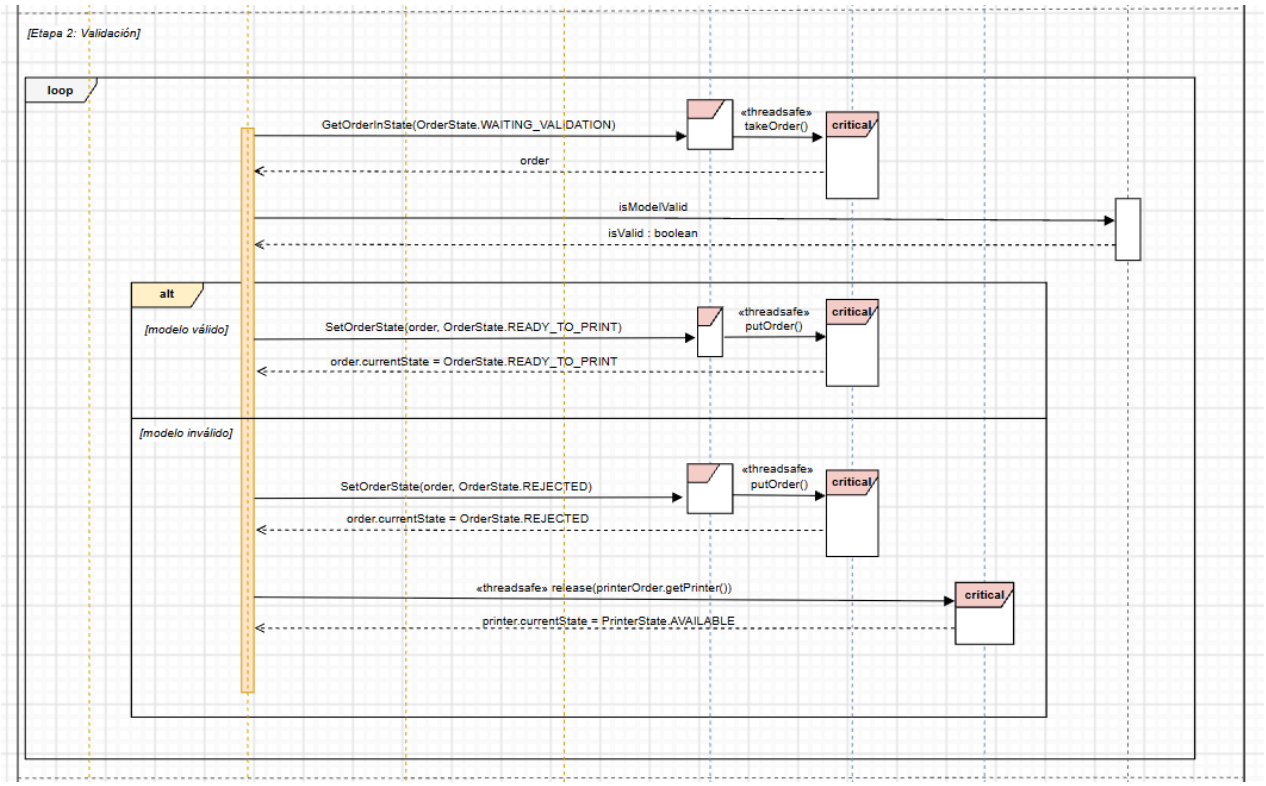
Por último, los recuadros en color verde (ENUMS) contienen los estados de las impresoras, de las órdenes, de los eventos y las etapas.

Diagrama de secuencia:

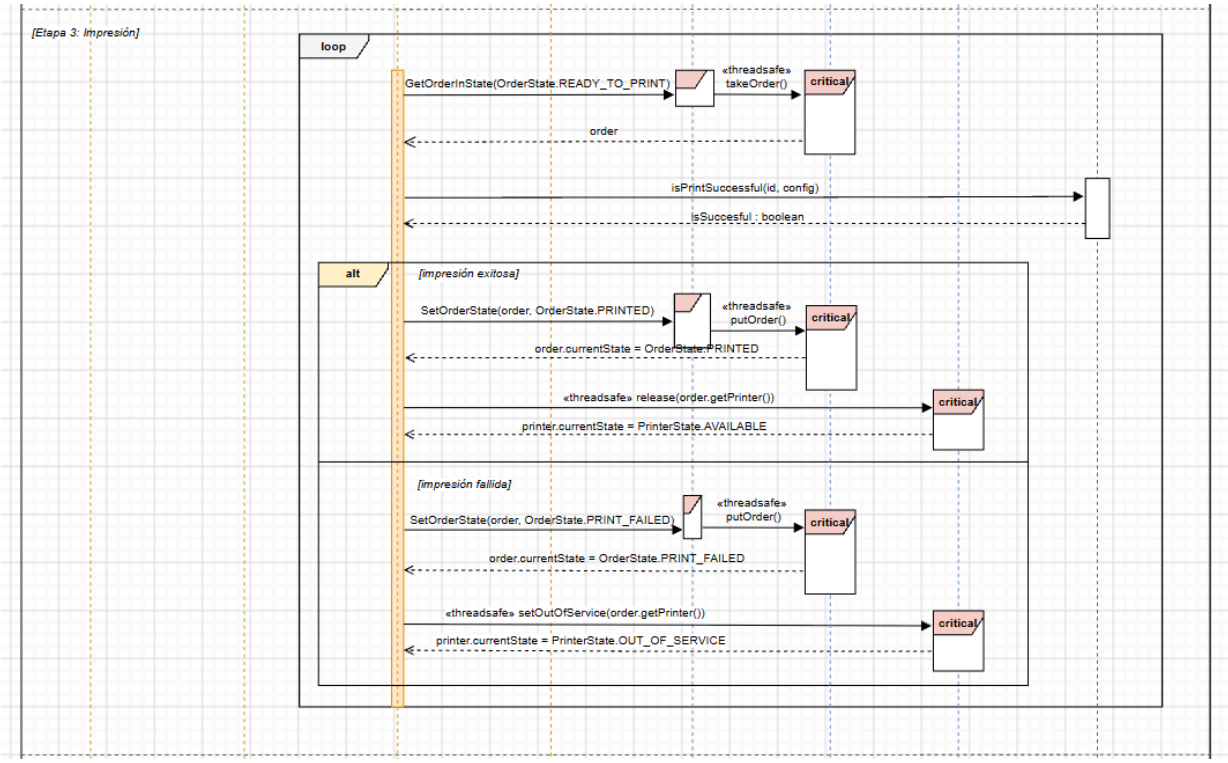
Para este diseño se modeló el núcleo de la concurrencia, el cual implementa el flujo de trabajo de los distintos hilos (Workers). Este diagrama detalla cómo las 4 etapas se ejecutan en paralelo mediante un bloque cooperativo (par), permitiendo que múltiples instancias de un mismo tipo de hilo operen simultáneamente sobre las órdenes de fabricación.



En esta primera etapa de asignación se toman los pedidos y se reserva la impresora. El hilo se bloquea en la región crítica de la cola compartida ejecutando de forma segura un método takeOrder() filtrando por las órdenes con estado CREATED. Una vez obtenido el pedido, el hilo invoca a PrinterFarm para reservar una impresora libre mediante reserve(order.getId()). Al confirmarse el recurso, se asocia la impresora a la orden (order.setPrinter) y el hilo cambia de forma segura el estado del pedido a WAITING_VALIDATION mediante un putOrder(). La exclusión mutua aquí garantiza que dos hilos de asignación distintos no intenten reservar la misma impresora.

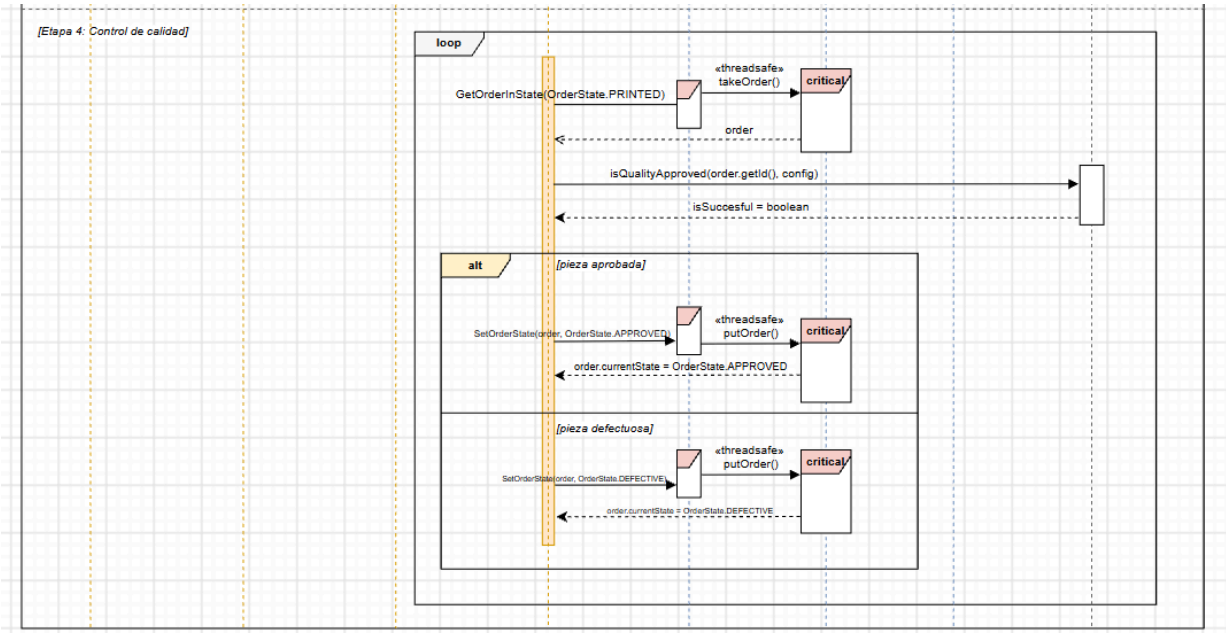


A partir de las segunda etapa, el recorrido empieza a recurrir a la clase OutcomeDecider dada por la cátedra, invocando de forma síncrona al método isModelValid(order.getId()), donde los hilos de validación buscan validar el modelo a imprimir de la órdenes en estado WAITING_VALIDATION. Si el resultado es exitoso, la orden cambia su estado a READY_TO_PRINT y se dirige a la cola de la siguiente etapa. Si no, la orden pasa a estado REJECTED. Al abortarse el proceso, este hilo es responsable de liberar la impresora previamente reservada invocando a release() en PrinterFarm, devolviendo la máquina al estado AVAILABLE para que otros hilos puedan usarla.



Es importante destacar que, con cada etapa aumentan la cantidad de métodos, instrucciones que se realizan, mientras que el manejo de la concurrencia en las secciones críticas se mantiene relativamente similar.

En la tercera etapa, los workers de impresión extraen pedidos en estado READY_TO_PRINT y se vuelve a consultar a la clase OutcomeDecider mediante isPrintSuccessful(id, config) para determinar el destino de la fabricación simulada. Si la impresión es exitosa, la orden se marca como PRINTED y se guarda en la cola correspondiente. Inmediatamente después, el hilo libera la impresora para que quede disponible (AVAILABLE) en la granja de impresión. Si no, el pedido se marca como PRINT_FAILED, y el hilo no solo libera la impresora, sino que también notifica a la granja que la máquina debe cambiar su estado a OUT_OF_SERVICE impidiendo que otros hilos la usen.



Por último, en la cuarta etapa se procesa de forma concurrente los elementos con estado PRINTED. El hilo invoca por última vez al OutcomeDecider de la cátedra mediante el método isQualityApproved(order.getId(), config). Si la pieza presenta fallas simuladas, se actualiza el estado a DEFECTIVE. Si no, se actualiza el estado a APPROVED.

Código:

A partir de estos diagramas, planificamos el código para conseguir el flujo concurrente entre los hilos. Los métodos más importantes, los cuales debían ser protegidos con una sección crítica, fueron los siguientes:

- **PrinterFarm:**

  Su sección crítica administra el uso de las impresoras, siendo sus métodos más importantes:

  - **reserve**
  - **release**
  - **setOutOfService**

  Estos métodos se manejan mediante la keyword **synchronized,** modificando los estados de las impresoras y el contador de la clase que lleva registro de las que se encuentran disponibles. Se utiliza, además, el método **wait** para los casos donde todas las impresoras están inutilizables, y el método **notifyAll** una vez liberada alguna impresora, notificando a los hilos en espera para que puedan continuar su recorrido.

```java
    public synchronized Printer reserve(PrintOrder order) throws InterruptedException {
        while (availableCount == 0) {
            wait();
        }
        Printer printer = findAvailable();
        printer.reserve(order.getId());
        availableCount--;
        return printer;
    }

    public synchronized void release(Printer printer) {
        printer.setState(PrinterState.AVAILABLE);
        availableCount++;
        notifyAll();
    }

    public synchronized void setOutOfService(Printer printer) {
        printer.setState(PrinterState.OUT_OF_SERVICE);
    }
```

- **OrderQueue:**

Esta clase es una cola segura de órdenes de impresión (de un mismo estado) para multihilos. Sus secciones críticas permiten que diferentes hilos agreguen y/o retiren órdenes de forma sincronizada, evitando condiciones de carrera al introducir un **key object** llamado **orderStateKey**. Si se da la situación en que todas las órdenes están siendo procesadas, se suspenden los hilos mediante el método **wait** hasta que la cola deje de estar vacía, siendo reactivados por el método **notifyAll** para que puedan retomar su objetivo. En caso de que se haya finalizado la etapa de la cola en cuestión, se retorna un null para finalizar sus workers.

```java
    public PrintOrder takeOrder() throws InterruptedException {
        synchronized (orderStateKey) {
            while (orderStateList.isEmpty()) {
                if (closed) {
                    return  null;
                }
                orderStateKey.wait();
            }
            return orderStateList.removeFirst();
        }
    }

    public void putOrder(PrintOrder printOrder) {
        synchronized (orderStateKey) {
            printOrder.incrementStageCount();
            orderStateList.add(printOrder);
            orderStateKey.notifyAll();
        }
    }

    public void Close() {
        synchronized (orderStateKey) {
            closed = true;
            orderStateKey.notifyAll();
        }
    }
```

- **Logger:**

Esta clase es un registrador centralizado y seguro para multihilos que almacena la simulación en archivos CSV. Sus métodos estáticos y sincronizados (con **synchronized**) previenen condiciones de carrera, asegurando que los hilos escriban de manera ordenada los eventos (**recordEvent**) y el balance final de cada orden (**recordElement**). Finalmente, el método **close** funciona como un mecanismo de cierre que garantiza la persistencia de todos los datos y la liberación de los recursos del sistema.

```
27      public static synchronized void init(Path outputDirectory, long simulationStartMillis) throws IOException {
28          Files.createDirectories(outputDirectory);
29          eventsWriter = Files.newBufferedWriter(outputDirectory.resolve("eventos.csv"), StandardCharsets.UTF_8);
30          elementsWriter = Files.newBufferedWriter(outputDirectory.resolve("elementos.csv"), StandardCharsets.UTF_8);
31          sequence = 0;
32          startMillis = simulationStartMillis;
33
34          write(eventsWriter, EVENTS_HEADER);
35          write(elementsWriter, ELEMENTS_HEADER);
36      }
37
38      public static synchronized void recordCreated(int orderId) {
39          writeEvent(Thread.currentThread().getName(), orderId, StageState.INITIALIZATION,
40                  EventState.ORDER_CREATED, null, OrderState.CREATED, null);
41      }
42
43      public static synchronized void recordEvent(Worker worker, int orderId, OrderState from,
44                                                  OrderState to, String printerId) {
45          writeEvent(Thread.currentThread().getName(), orderId, worker.getStage(),
46                  EventState.ORDER_STATE_CHANGED, from, to, printerId);
47      }
48
49      public static synchronized void recordElement(int orderId, OrderState finalState, String printerId,
50                                                    int assignmentCount, int validationCount,
51                                                    int printingCount, int qualityControlCount) {
52          String line = String.join(";",
53                                    Integer.toString(orderId),
54                                    finalState.name(),
55                                    nullToEmpty(printerId),
56                                    Integer.toString(assignmentCount),
57                                    Integer.toString(validationCount),
58                                    Integer.toString(printingCount),
59                                    Integer.toString(qualityControlCount));
60          write(elementsWriter, line);
61      }
62
63      public static synchronized void close() throws IOException {
64          try {
65              if (eventsWriter != null) eventsWriter.close();
66          } finally {
67              if (elementsWriter != null) elementsWriter.close();
68              eventsWriter = null;
69              elementsWriter = null;
70          }
71      }
```

Las clases a continuación sobreescriben el método proveniente de su clase padre, **Worker**.

- **AssignWorker:**

  Esta clase es la encargada de la ETAPA 1 y es instanciada por 3 hilos, donde cada uno debe:

  1. Tomar una orden en estado CREATED (con **getOrderInState**).
  2. Obtener una impresora disponible (en **Reserve**).
  3. Reservarla de manera exclusiva (con **Reserve**).
  4. Incrementar su contador de usos (en **SetOrderState**).
  5. Asociarla con la orden (con **SetOrderState**).
  6. Cambiar la orden a WAITING_VALIDATION (con **SetOrderState**).

```
16        @Override
17        protected void process() throws InterruptedException {
18            PrintOrder printOrder = orderSource.getOrderInState(OrderState.CREATED);
19
20            if (printOrder == null) {
21                Thread.currentThread().interrupt();
22                return;
23            }
24
25            Printer printer = printerFarm.reserve(printOrder);
26            printOrder.setPrinter(printer);
27            orderSource.SetOrderState(printOrder, OrderState.WAITING_VALIDATION, this);
28        }
```

- **ValidationWorker:**

  Esta clase es la encargada de la ETAPA 2 y es instanciada por 2 hilos, donde cada uno debe:

  1. Tomar una orden en estado WAITING_VALIDATION (con **getOrderInState**).
  2. Validar su modelo (con **isModelValid**).
  3. Actualizar su estado (con **SetOrderState**).
  4. Actualizar el estado de la impresora de ser necesario (con **release**).

```
22        @Override
23        protected void process() throws InterruptedException {
24            PrintOrder printOrder = orderSource.getOrderInState(OrderState.WAITING_VALIDATION);
25
26            if (printOrder == null) {
27                Thread.currentThread().interrupt();
28                return;
29            }
30
31            if(OutcomeDecider.isModelValid(printOrder.getId(), simulationConfig)) {
32                orderSource.SetOrderState(printOrder,OrderState.READY_TO_PRINT, this);
33            } else {
34                orderSource.SetOrderState(printOrder,OrderState.REJECTED, this,
35                                          () -> printerFarm.release(printOrder.getPrinter()));
36            }
37        }
```

- **PrintingWorker:**

  Esta clase es la encargada de la ETAPA 3 y es instanciada por 3 hilos, donde cada uno debe:

  1. Tomar una orden en estado READY_TO_PRINT (con **getOrderInState**).
  2. Determinar su resultado (con **isPrintSuccessful**).
  3. Actualizar su estado (con **SetOrderState**).
  4. Actualizar el estado de la impresora dependiendo del resultado (con **release** o **setOutOfService**).

```
25      @Override
26      protected void process() throws InterruptedException {
27          PrintOrder readyToPrintOrder = orderSource.getOrderInState(OrderState.READY_TO_PRINT);
28          if (readyToPrintOrder == null) {
29              Thread.currentThread().interrupt();
30              return;
31          }
32
33          if (OutcomeDecider.isPrintSuccessful(readyToPrintOrder.getId(), this.config)) {
34              orderSource.SetOrderState(readyToPrintOrder, OrderState.PRINTED, this,
35                                        () -> printerFarm.release(readyToPrintOrder.getPrinter()));
36          } else {
37              Printer printer = readyToPrintOrder.getPrinter();
38              orderSource.SetOrderState(readyToPrintOrder, OrderState.PRINT_FAILED, this,
39                                        ()->printerFarm.setOutOfService(printer));
40          }
41      }
```

- **QualityWorker:**

  Esta clase es la encargada de la ETAPA 4 y es instanciada por 2 hilos, donde cada uno debe:

  1. Tomar una orden en estado PRINTED (con **getOrderInState**).
  2. Determinar su resultado (con **isApproved** el cual llama a **isQualityApproved**).
  3. Actualizar su estado (con **SetOrderState**).

```
21      @Override
22      protected void process() throws InterruptedException {
23          PrintOrder printedOrder = orderSource.getOrderInState(OrderState.PRINTED);
24
25          if (printedOrder == null) {
26              Thread.currentThread().interrupt();
27              return;
28          }
29
30          if (isApproved(printedOrder)) {
31              orderSource.SetOrderState(printedOrder, OrderState.APPROVED, this);
32          } else {
33              orderSource.SetOrderState(printedOrder, OrderState.DEFECTIVE, this);
34          }
35      }
```

Tests:

Implementamos los tests para comprobar que la concurrencia ocurre de forma correcta y no se corrompen o modifican los datos al terminar el programa. A continuación mostramos los tests y sus resultados:

```java
@DisplayName("reserve: la impresora queda RESERVED, asociada a la orden y con un uso")
void reserveAsociaLaOrden() throws InterruptedException {
    PrinterFarm farm = new PrinterFarm( rows: 2,  columns: 2);

    Printer printer = farm.reserve(new PrintOrder( id: 7));

    assertEquals(PrinterState.RESERVED, printer.getCurrentState());
    assertEquals( expected: 7, printer.getAssignedOrderId());
    assertEquals( expected: 1, printer.getUsageCount());
    assertEquals( expected: 1, farm.totalOrders());
}
```

```java
@DisplayName("release: la impresora vuelve a AVAILABLE, sin orden, y se puede volver a reservar")
void releaseLaDejaReutilizable() throws InterruptedException {
    PrinterFarm farm = new PrinterFarm( rows: 1,  columns: 1);
    Printer printer = farm.reserve(new PrintOrder( id: 1));

    farm.release(printer);

    assertEquals(PrinterState.AVAILABLE, printer.getCurrentState());
    assertNull(printer.getAssignedOrderId());

    Printer again = farm.reserve(new PrintOrder( id: 2));
    assertSame(printer, again);
    assertEquals( expected: 2, again.getAssignedOrderId());
    assertEquals( expected: 2, again.getUsageCount());
}
```

```java
@DisplayName("reserve: pasa a RESERVED, suma un uso y guarda la orden")
void reserveMarcaReservadaYGuardaOrden() {
    Printer printer = new Printer( row: 0,  col: 0);

    printer.reserve( orderId: 42);

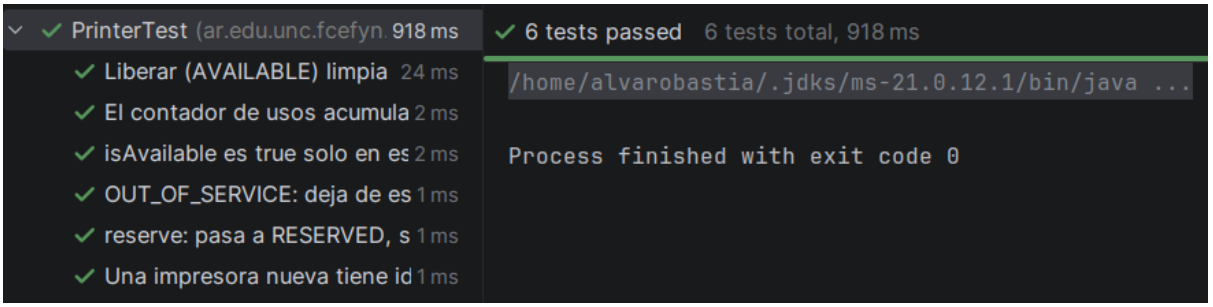
    assertEquals(PrinterState.RESERVED, printer.getCurrentState());
    assertFalse(printer.isAvailable());
    assertEquals( expected: 1, printer.getUsageCount());
    assertEquals( expected: 42, printer.getAssignedOrderId());
}
```

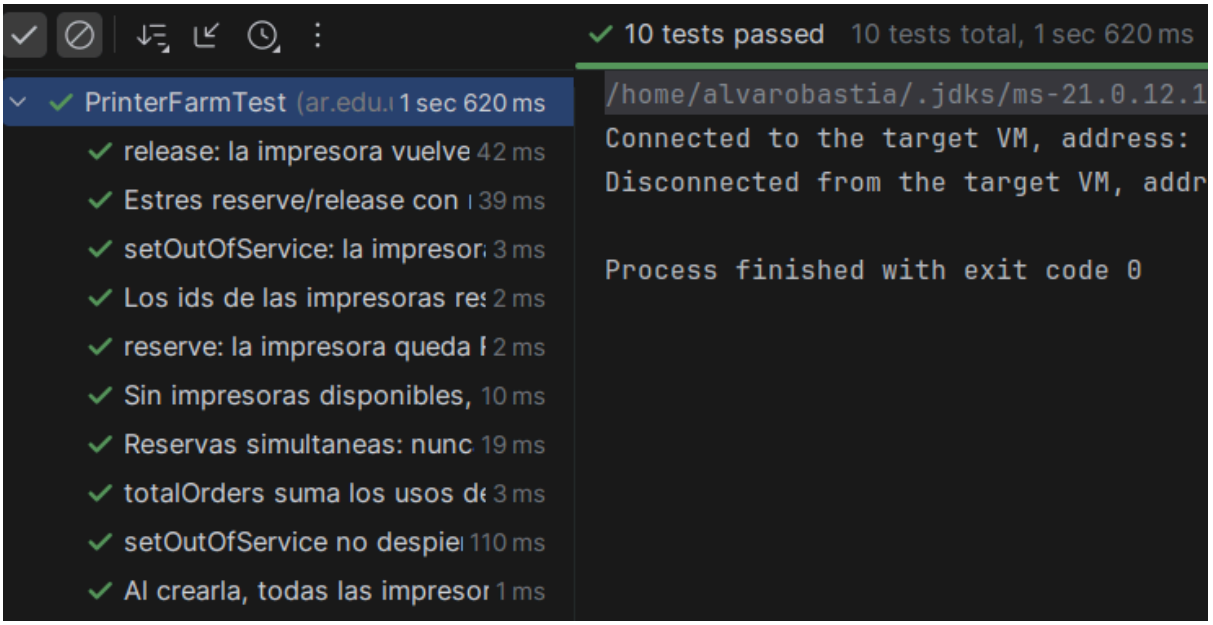
```java
@DisplayName("Liberar (AVAILABLE) limpia la orden asignada pero conserva el contador de usos")
void liberarLimpiaOrdenYConservaUsos() {
    Printer printer = new Printer( row: 0,  col: 0);
    printer.reserve( orderId: 1);

    printer.setState(PrinterState.AVAILABLE);

    assertTrue(printer.isAvailable());
    assertNull(printer.getAssignedOrderId());
    assertEquals( expected: 1, printer.getUsageCount());
}
```

Los cuatro tests anteriores son de la clase PrinterFarm y Printer, a continuación se muestran los resultados de todos los tests ejecutados:





Resultados:

Luego de que el programa principal se termine de ejecutar, se guardan en la carpeta resultados tres archivos importantes:

elementos.csv:

Este mismo escribe en un archivo de texto lo que ocurre dentro de cada cambio de estado de las órdenes, a continuación una imagen con el archivo escrito.

```
1   sequence;elapsedMs;thread;orderId;stage;event;fromState;toState;printer
2   1;2;main;1;INITIALIZATION;ORDER_CREATED;;CREATED;
3   2;2;main;2;INITIALIZATION;ORDER_CREATED;;CREATED;
4   3;2;main;3;INITIALIZATION;ORDER_CREATED;;CREATED;
```

```
799 798;2794;printing-3;47;PRINTING;ORDER_STATE_CHANGED;READY_TO_PRINT;PRINTED;P-1-14
800 799;2794;assignment-3;168;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-1-14
801 800;2794;quality-2;47;QUALITY_CONTROL;ORDER_STATE_CHANGED;PRINTED;APPROVED;P-1-14
802 801;2794;validation-2;48;VALIDATION;ORDER_STATE_CHANGED;WAITING_VALIDATION;READY_TO_PRINT;P-1-15
803 802;2845;assignment-1;169;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-6-4
804 803;2845;assignment-2;170;ASSIGNMENT;ORDER_STATE_CHANGED;CREATED;WAITING_VALIDATION;P-6-5
```

Se pueden observar las mismas etapas y estados por las que pasan las distintas órdenes con sus respectivas impresoras. La línea número 1 representa el header con la información que está entre cada ‘;’.

eventos.csv:

Este archivo deja asentado lo que ocurre con todas las órdenes al final de la ejecución, a continuación una imagen con el archivo escrito.

```
1   orderId;finalState;printer;assignmentCount;validationCount;printingCount;qualityControlCount
2   1;APPROVED;P-0-0;1;1;1;1
```

```
6   5;REJECTED;P-0-2;1;1;0;0
7   6;APPROVED;P-0-3;1;1;1;1
8   7;REJECTED;P-0-4;1;1;0;0
9   8;PRINT_FAILED;P-0-5;1;1;1;0
```

El mismo muestra el estado final que quedó cada orden, su impresora asociada durante el proceso (si es que llegó a asociarse a una) y los valores totales de asignación, validación, printeo y control de calidad.

resumen.properties:

Este archivo muestra un mini pantallazo general en números de los datos más importantes.

```
1   totalOrders=500
2   processedOrders=500
3   approvedOrders=359
4   rejectedOrders=71
5   printFailedOrders=45
6   defectiveOrders=25
7   durationMillis=30213
8   allThreadsTerminated=true
9   remainingIntermediateOrders=0
```

Una estadística única por cada programa corrido, este puede variar gracias a los condicionales dados por la cátedra en OutcomeDecider.

Conclusión:

Primer trabajo práctico de la materia donde se adentra de lleno a la concurrencia de múltiples hilos con múltiples etapas paralelas. Logramos aprender cómo diseñar la concurrencia en un diagrama de flujos, entender que clases realizan secciones críticas a partir de los métodos señalados en el diagrama de clases, generamos distintas formas de manipular la concurrencia con los bloques synchronized, aprendimos a cómo explicar la elección de la forma elegida para batallar estos problemas y todo esto mientras se trabajaba en equipo a través de un repositorio en github.